

Task 2: Memory Management Simulation

Analysis Report — ST5004CEM Operating Systems and Security

Overview

task2_memory.cpp simulates virtual memory paging with a configurable number of frames (set to 3 for this analysis) and two page replacement algorithms: First-In-First-Out (FIFO) and Least Recently Used (LRU). The same reference string is run through both so their behaviour can be compared directly.

Test Setup

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (12 page requests)

Number of frames: 3

Results

Algorithm	Page Faults	Page Hits	Hit Ratio
FIFO	9	3	25.0%
LRU	10	2	16.7%

How Each Algorithm Behaves

FIFO keeps a simple arrival-order queue of the pages currently in memory. When a fault occurs and memory is full, whichever page has been resident the longest is evicted, regardless of how recently it was actually used.

LRU keeps the pages ordered by recency of use. On a hit, the page is moved to the 'most recently used' end. When a fault occurs and memory is full, the page at the 'least recently used' end is evicted.

Where the Two Algorithms Diverge

Both algorithms behave identically for the first seven requests (1,2,3,4,1,2,5), reaching the same state: frames = [1, 2, 5], having faulted every time. The difference appears at request 3 (the eighth request):

- FIFO evicts page 1, because it is the oldest page in the queue by insertion order (1 was loaded first, even though it was just re-used as a hit two steps earlier).
- LRU evicts page 5, because although 1 and 2 were just hit (making them 'recently used'), 5 has not been touched since it was loaded and is therefore the least recently used page.

This single decision cascades through the rest of the reference string: LRU's choice to evict page 5 means requests 3, 4 and 5 all fault again at the end of the run (evicting 1, then 2), giving LRU three additional faults compared with FIFO over that stretch.

Discussion

The result that FIFO produced fewer faults than LRU (9 vs 10) is worth highlighting rather than treating as an anomaly to hide: it demonstrates that no single page replacement algorithm is optimal for every access pattern. LRU's recency heuristic is generally a good predictor of future use, but for this particular reference string it evicted a page (5) shortly before that page would have been needed again, triggering a chain of extra faults. FIFO's simpler 'oldest in, oldest out' rule happened to retain the right pages for this specific sequence purely by insertion order, not because it reasoned about usage at all.

This is consistent with well-documented behaviour of page replacement algorithms in operating systems: algorithm choice should be evaluated against the actual access patterns of the workload it will serve, since a theoretically 'smarter' algorithm (LRU tracks more information than FIFO) does not guarantee better results on every input.

Conclusion

Both FIFO and LRU were implemented, tested against the same 12-request reference string with 3 frames, and their full step-by-step behaviour (including every hit, fault, and eviction) was logged for verification. FIFO achieved a 25.0% hit ratio and LRU achieved 16.7% on this particular workload, illustrating that replacement algorithm performance is workload-dependent rather than universally ranked.